Maximizing Customer Lifetime Value using Stacked Neural Networks: An Insurance Industry Application

Gaddiel Desirena †, Armando Díaz†, Jalil Desirena†, Ismael Moreno† and Daniel Garcia†,

†Relativity6 Inc.

***Abstract*—This paper proposes a recommender system based on two-stage neural network architecture that maximizes Customer Lifetime Value (CLV). The Stage-I neural network uses a self-attention mechanism and a Collaborative Metric Learning (CML) to generate product recommendations. The Stage-II neural network uses a neural network-based survival analysis to infer insurance product recommendations that maximize customer lifetime. The proposed stacked neural network model can be used as a generative model to explore different cross-sell scenarios. The applicability of the proposed recommendation system is evaluated using transactional data from an Australian insurance company. We validated our results against a state of the art self-attention recommendation system, successfully extending its functionality to include lifetime value.**

***Index Terms*—Recommender systems, Neural Networks, Customer Lifetime Value, Self-Attention Mechanism, Customer retention, Insurance product recommendation**

I. Introduction

The growing number of churning customers in the insurance industry has prompted insurers to look for new customer retention strategies to keep and engage their current customers and prevent declining revenues. Efforts in customer retention have a larger effect on profits than actively acquiring new customers. It is common for insurers to lose up to 15% of their clients each year. Reducing defections by just 5%, would increase the average company's growth rate by more than 50% [1]. One approach to solve the problem involves estimating customer value to identify which customer relationships will increase and which will decrease in value or revenue for insurers. For this aim, Customer Lifetime Value (CLV), which is the present value of all future profits coming from a customer over its lifetime with a company, serves as a valid metric to evaluate customer value. Customer Lifetime Value has become a topic of interest in the scientific community as an option to address the problem of customer retention and churn prediction [2]. CLV models rely on customer income at every stage of their life cycle to make good estimations, using an array of techniques from survival analysis [3], [4] and data mining [2], [5]. In [6], a Pareto/NBD model is used to predict CLV. The first component of the model predicts the number of future transactions. Then, another component provides an estimated profit for each of the predicted future transactions. After that, CLV is computed with the outputs of these components. These methods focus on estimating CLV rather than optimizing it.

† Gaddiel Desirena, Armando Díaz, Jalil Desirena, Ismael Moreno and Daniel Garcia are with the Relativity6 Inc. Computer Science & Artificial Intelligence Lab, Santa Rita 925, Chapalita Oriente, CP 45040, Zapopan, Jalisco, México { gaddiel, armando, jalil, ismael, daniel}@relativity6.com

Recommender systems (RSs), which have been very profitable for the digital business, can play a bigger role in the insurance industry specifically. Currently some insurance companies recommend policies to a client for cross-sell or up-sell opportunities with the help of recommender systems. A large variety of methods are used to obtain recommendations such as Collaborative Filtering (CF), Content filtering and hybrids [7], [8], [9]. RSs consider user preferences, product data and transactional data to suggest next best items for purchase. In recent years, attention-based recommender systems have been proposed extensively [10], [11]. In [12] authors propose an attention-based user behavior modeling framework called ATRank for recommendation tasks. In [13] and [14], self-attention mechanisms are used to infer the item-item relationship from users' historical interactions for the retail industry. Considerable progress has been made in deep learning techniques applied to recommendation tasks. One common approach is the Convolutional Neural Network (CNN) that can be used to extract features from text, video, and audio datasets [15] [16]. Ansell et al. [17], apply life-stage segmentation and survival analysis based on Cox's linear regression to identify cross-sell opportunities in 10,979 UK customers of a large insurance company. They show that it is possible to identify which customers are more likely to re-purchase and to predict the time frame in which they will re-purchase. In this paper, we propose a recommender system that maximizes CLV for an insurance company, which outputs risk and survival curves for the top n recommended products for each customer. The key contributions of our work are the following: a) A method that captures both product recommendations and CLV optimization for the insurance industry. It considers product features to infer an increase or decrease of CLV, easing the exploration of different products that might extend or reduce lifetime and increase or decrease profits. b) The method captures the non-linear relationship between recommended products and Customer Lifetime Value, resulting in more realistic predictions.

The rest of this paper is structured as follows. Section II shows the data processing steps. Section III describes Stage I of the method, discussing the Self-Attention Recommender system. Section IV details Stage II where a CLV model via non-linear survival analysis is used. Section V shows the proposed method to maximize CLV via recommended system. Section VI goes on to show Experimental Results. Section VII presents the Conclusion, and finally the Section Appendix shows key mathematical ideas in the proposal.

II. Data Processing

We will now show how the insurance dataset is cleaned and transformed for use in our recommender system. Consider a set of users and items denoted by $\mathcal{U} = \{u_1, \ldots, u_m\}$ and

978-1-7281-4550-1/19/$31.00 ©2019 IEEE
DOI 10.1109/ICMLA.2019.00101

Authorized licensed use limited to: Cambridge Institute of Technology - Bengaluru. Downloaded on March 12,2026 at 09:24:14 UTC from IEEE Xplore. Restrictions apply.

$\mathcal{S} = \{s_1, \ldots, s_n\}$. Let $\boldsymbol{X} = \boldsymbol{X}_p \cup \boldsymbol{X}_u$ be the dataset that contains item features and user features, where $\boldsymbol{X}_p \in \mathcal{R}^{(n\times m)\times d}$, $\boldsymbol{X}_u \in \mathcal{R}^{(n\times m)\times f_u}$ and f_u represents the number of user features. Let o_{end} be the cutoff time for our dataset $\mathcal{X}$. Each user $u_i \in \mathcal{U}$ has a subscription interval denoted by $I^{u_i}_{s_j} = [t^{u_i}_{s_j}\ o^{u_i}_{s_j}]$ if the user has unsubscribed at o_{end} or $I^{u_i}_{s_j} = [t^{u_i}_{s_j}\ o_{end}]$ in the case he is still subscribed at o_{end}. The subscription and unsubscription time of user u_i to item s_j is denoted by $t^{u_i}_{s_i}$ and $o^{u_i}_{s_j}$ respectively. The time period of the subscription is $|I^{u_i}_{s_j}| = o_{end}\cdot(1-e^{u_i}_{s_j}) + o^{u_i}_{s_j}\cdot e^{u_i}_{s_j} - t^{u_i}_{s_j}$. The parameter $e^{u_i}_{s_j}$ is the subscription status for user u_i to item s_j. The values $e^{u_i}_{s_j}$ will be 1 in the case it is subscribed and 0 if the user has no active subscription at time o_{end}. A purchase history of each user u_i is represented by the 3-tuple $\gamma_{u_i} = (I^{u_i}_{s_j}, \mathcal{S}_{u_i}, \boldsymbol{X}_{u_i})$ where $\mathcal{S}_{u_i} = \{s^i_1, \ldots, s^i_{|\mathcal{S}_{u_i}|}\}$ represents a sequence of items in the order that the user u_i purchased the items before o_{end}. The *purchase history matrix* $\boldsymbol{X}_{u_i} \in \mathcal{R}^{l\times d}$ (obtained from $\boldsymbol{X}_p$) has data containing summary features of the user u_i l-th purchase up to time O_{end}. The first purchase $\boldsymbol{x}^1_{u_i} = [x_{1,1}, \ldots, x_{1,d}]$ is a d-dimensional embedding vector that has d features of an item in set $\mathcal{S}$. The *purchase history matrix* is built from all the l d-dimensional embedding vectors: $\boldsymbol{X}_{u_i} = \begin{bmatrix} \boldsymbol{x}^1_{u_i} \\ \vdots \\ \boldsymbol{x}^l_{u_i} \end{bmatrix}$.

III. Stage I: Self-Attention Recommender System

The recommender system uses a self-attention model to deal with sequential data and a preference model that uses Collaborative Metric Learning (CML) to tackle users' item preferences [18]. The objective is to predict the items that the user u_i will purchase next given their purchase history coming from matrix $\boldsymbol{X}_{u_i}$.

A. Self-Attention Mechanism

The Self-Attention model maps a query and a set of key-value pairs to an output. The query, keys, values, and output are all vectors [19]. This allows us to calculate the global dependencies between users and items. To construct the Self-Attention model, we first project the purchase history matrix $\boldsymbol{X}_{u_i}$ to the spaces $\mathcal{F}(\boldsymbol{X}_{u_i})$ and $\mathcal{G}(\boldsymbol{X}_{u_i})$, where $\mathcal{F}(\boldsymbol{X}_{u_i}) = \boldsymbol{X}_{u_i}\boldsymbol{W}_{\mathcal{F}}$, $\mathcal{G}(\boldsymbol{X}_{u_i}) = \boldsymbol{X}_{u_i}\boldsymbol{W}_{\mathcal{G}}$ and matrices $\boldsymbol{W}_{\mathcal{F}}, \boldsymbol{W}_{\mathcal{G}} \in \mathcal{R}^{d\times d}$ become the learned weights for query and key respectively, which are implemented as 1x1 convolutions. We calculate the dot products of the query with all keys in $\boldsymbol{\Pi} = \mathcal{F}(\boldsymbol{X}_{u_i})\mathcal{G}(\boldsymbol{X}_{u_i})^T$, where $\boldsymbol{\Pi} = (\pi_{i,j}) \in \mathcal{R}^{d\times d}$. Then, the attention matrix $\boldsymbol{\Psi}_{u_i} = (\psi_{i,j}) \in \mathcal{R}^{l\times l}$ is computed, and now contains the similarities between the l items of user u_i. The similarity between the i-th item and the j-th item is denoted by $\psi_{i,j}$ and obtained by computing the softmax of the attention scores $\pi_{i,j}$ as follows: $\psi_{i,j} = softmax(\pi_{i,j})$. The output $\boldsymbol{\zeta_{u_i}} = (\zeta_1, \ldots, \zeta_j, \ldots, \zeta_l) \in \mathcal{R}^l$ of the Self-Attention module is obtained from the mean of the product of the attention matrix $\boldsymbol{A} = (a_{i,j}) \in \mathcal{R}^{l\times l}$ and the purchase history matrix (i.e., $\boldsymbol{A} = \boldsymbol{\Psi}_{u_i}\boldsymbol{X}_{u_i} \in \mathcal{R}^{l\times l}$). Thus each $\zeta_j \in \boldsymbol{\zeta_{u_i}}$ is computed as: $\zeta_j = \frac{1}{l}\sum_{i=1}^{l} a_{i,j}$ [18]. Following the transformation of $\boldsymbol{X}_{u_i}$, we compute a geometric sequence of timescales so we can add sinusoidals of different frequencies to the input. The time embedding function $\Gamma(t, 2i)$ consists of two sinusoidal signals defined as follows, where t is the time step, and i the dimension: $\Gamma(t, 2i) = sin(t/10000^{2i/\alpha})$ and $\Gamma(t, 2i+1) = cos(t/10000^{2i/\alpha})$.

B. Preference model

We encode the user-item interaction matrix to latent factors. Let the users' and items' latent factors be the matrices $\boldsymbol{U}_i \in \mathcal{R}^{m\times d}$ and $\boldsymbol{I}_j \in \mathcal{R}^{n\times d}$. To obtain the minimum distance between users and items a scoring function is used, which finds the Euclidean distance between each user and item interaction: $Score(\boldsymbol{U}_i, \boldsymbol{I}_j) = ||\boldsymbol{U}_i - \boldsymbol{I}_j||^2_2$, where $||\cdot||^2_2$ is the L2 norm of the matrix $\boldsymbol{U}_i - \boldsymbol{I}_j$. If user u_i likes item s_j the distance will be small and large otherwise.

C. Objective function

Let $\mathcal{S}^{l+1}_{u_i}$ be the set of k predicted items s^k_{l+1} for user u_i that will be recommended after time o_{end}. The cost function used to train the model minimizes Euclidean distance as in [18], is computed as follows: $\boldsymbol{y}^{l+1}_{u_i} = ||\boldsymbol{U}_i - \boldsymbol{I}_{l+1}||^2_2 + ||\boldsymbol{\zeta}_{u_i} - \boldsymbol{x}^{l+1}_{u_i}||^2_2$. Note that $\boldsymbol{I}_{l+1}$ and $\boldsymbol{x}^{l+1}_{u_i}$ are the embedding vectors for the next item $s^k_{l+1} \in \mathcal{S}^{l+1}_{u_i}$. Thus the purchase history matrix for the next $l+1$ purchase is denoted by $\boldsymbol{X}_{u_i} = [\boldsymbol{X}_{u_i}; \boldsymbol{x}^{l+1}_{u_i}]$. In order to predict a set of next items, the pairwise ranking loss is minimized to learn the model parameters. The pairwise ranking loss is defined as: $\mathcal{L}(\Phi) = \sum_{(u_i,s_j)\in\mathcal{S}^+}\sum_{(u_i,s_k)\notin\mathcal{S}^+} \max\left(0, \boldsymbol{y}^j_{u_i} + \Delta - \boldsymbol{y}^k_{u_i}\right) + \lambda||\boldsymbol{\Phi}||^2_2$, where $\boldsymbol{\Phi} = \{\boldsymbol{X}, \boldsymbol{I}_j, \boldsymbol{U}_i, \boldsymbol{W}_{\mathcal{F}}, \boldsymbol{W}_{\mathcal{G}}\}$ represents the model parameters, and $\mathcal{S}^+$ denotes the set of next k items to recommend to user u_i [18]. The set of items the user does not prefer is denoted by $\mathcal{S}^-$ ($\mathcal{S}^- = \mathcal{S} - \mathcal{S}^+$), Δ represents the difference between the preferred items and unpreferred items. The parameter λ is the regularization parameter. We use L_2 regularization to control the model complexity.

IV. Stage II: Customer Lifetime Value using Non-Linear Survival Analysis

A. Customer Lifetime Value

The CLV function CLV_{u_i} captures all the purchases that a customer u_i will make, for the n items that the company sells. As in the model proposed in [20], we calculate customer u_i CLV before cutoff o_{end} from the sum of the net cash flows $\mathcal{C}^{u_i}_{s_j}(t)$ obtained per purchased item s_j and discounted at an assumed constant rate d at time t. The discount factor d is usually set based on business knowledge. Given these parameters, we can compute the CLV function as follows: $CLV_{u_i} = \sum_{j=1}^{l} \mathcal{C}^{u_i}_{s_j}(t+j)\cdot(1+d)^{-j}$, where the net cash flow is $\mathcal{C}^{u_i}_{s_j}(t) = \varepsilon_j \cdot S^{u_i}_{s_j}(t)$, here ε_j corresponds to the marginal profit for product s_j. The total value coming from an active customer (CLV_{u_i}) allows us to calculate CLV for customer u_i at time t for the q products. In order to estimate $S^{u_i}_{s_j}(t)$ survival analysis is used.

B. Survival Analysis

Survival Analysis estimates the time until an event occurs. It comprises two elements: time effect and individual effect. The first is described as a survival function shared among the whole population. The second describes the difference of individuals from a base point in terms of the covariates. Combining these two functions makes it possible to describe quantitatively the probability of an event taking place at time t [4].

1) Survival Hazard Function: The survival hazard function is used to represent the probability that time of the event of interest occurs later than a specified time t. The survival function is represented by $S(t) = P(T \geq t)$. The Hazard function $h(t)$ represents the likelihood that the event occurs at time t given that no event has occurred before time t. Formally, the Hazard

Authorized licensed use limited to: Cambridge Institute of Technology - Bengaluru. Downloaded on March 12,2026 at 09:24:14 UTC from IEEE Xplore. Restrictions apply.

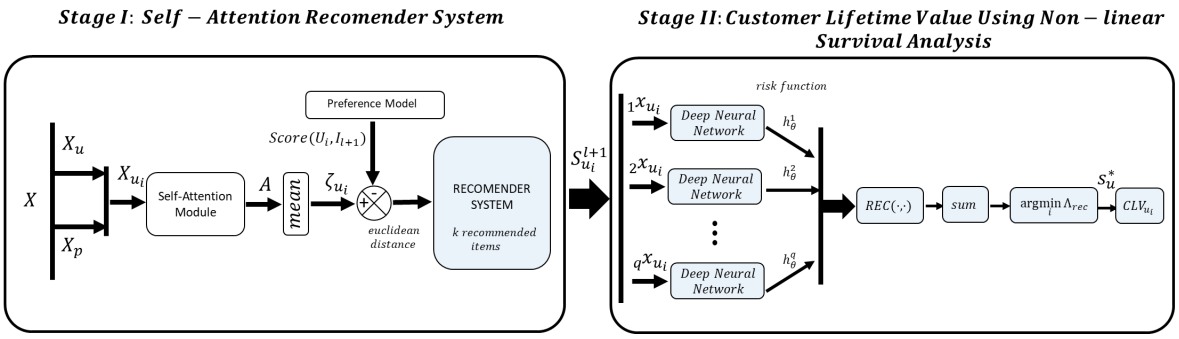

Fig. 1. Proposed stacked neural network. Stage I: Self-attention recommender system. Stage II: Customer lifetime value using non-linear survival.

function is defined as $h(t) = \lim_{\Delta t \to 0} \frac{F(t+\Delta t)-F(t)}{\Delta t \cdot S(t)} = \frac{f(t)}{S(t)}$, where $F(t) = 1 - S(t)$ is the cumulative distribution function and $f(t)$ is the density function $f(t) = F'(t)$. Note that, $f(t) = -S'(t)$. The cumulative Hazard function $H(t)$ is expressed as $H(t) = \int_0^t h(u)du$. Thus, the survival function can be rewritten as $S(t) = exp(-H(t))$. To estimate the survival hazard the Cox model is widely used due to its efficiency in linear regression analysis of survival data.

2) Neural network for survival analysis: In order to capture non-linear relationships of items and customers we propose a non-linear neural network based survival analysis. The network predicts the covariates of the Hazard rate parameterized by the weights θ. The output of the network is a single node with linear activation that estimates the log risk function in the Cox model [4]. The network is trained by setting the objective function to the average negative log partial likelihood of $h_\theta(\boldsymbol{x}^p_{u_i})$ with L_2 regularization: $l(\theta) = -\frac{1}{n_e}\sum_{q:e_q=1}\left(h_\theta(\boldsymbol{x}^q_{u_i}) - log\sum_{p\in\mathcal{R}(t^{u_i}_{s_q})} e^{h_\theta(\boldsymbol{x}^p_{u_i})}\right) + \lambda \cdot ||\theta||^2_2$, where n_e is the number of users with an observable event, and the set $\mathcal{R}(t) = \{q : t^{u_i}_{s_q} \geq t\}$ is the set of users still at risk of failure at time t. The parameter λ is the $\mathcal{L}_2$ regularization parameter. In order to find the weights of the network, the gradient descended method is used to minimized $l(\theta)$.

V. Maximizing CLV with Recommender Systems

In Section III the set $\mathcal{S}^{l+1}_{u_i}$ of recommended items was obtained. In this section we propose a final recommendation for user u_i that maximizes CLV (the best item s^q_{l+1} that minimizes the risk function). After estimating the hazard functions for the items $s_j \in \mathcal{S}^{l+1}_{u_i}$ based on the neural network, the survival functions $S^{u_i}_{s_j}(t)$ are obtained. Fig. 1 depicts the stacked neural network architecture of the recommender system. It follows two stages. Stage I is the self-attention recommender system. Stage II is the customer lifetime value using non-linear survival analysis. The output of Stage I is the set $\mathcal{S}^{l+1}_{u_i}$ of the k predicted items, $\mathcal{S}^{l+1}_{u_i}$ is used as an input to Stage II and it calculates the maximum CLV_{u_i} function. There are two ways of arriving at the final recommendation. In the first method we estimate a set of CLV functions CLV_{rec} for the set $\mathcal{S}^{l+1}_{u_i}$, and find the argmax of CLV_{rec} to obtain the final recommended item $s^*_{u_i}$. The second approach uses an anti symmetric matrix REC to represent the relationship between the different items in the set $\mathcal{S}^{l+1}_{u_i}$ and obtain the final recommendation item $s^*_{u_i}$.

To simplify the notation, we use s_j instead of s^j_{l+1}, which corresponds to one item of the q recommended items at the $l+1$ purchase. For each item $s_j \in \mathcal{S}^{l+1}_{u_i}$ the purchase history matrix $\boldsymbol{X}_{(u_i,s_j)} = [\boldsymbol{X}_{u_i}; \boldsymbol{x}^{l+1}_{(u_i,s_j)}]$ is calculated. Then, $\boldsymbol{x}^{l+1}_{(u_i,s_j)}$ is given as an input to the network. The Hazard function obtained by the network is: $h_{u_i}(t; \boldsymbol{X}_{(u_i,s_j)}|s_j \in \mathcal{S}^{l+1}_{u_i}) = \lambda_0 \cdot e^{h^j_\theta(\boldsymbol{X}_{(u_i,s_j)})}$, where $h^j_\theta(X_{(u_i,s_j)})$ represents the risk function for user u_i when they purchase the item s_j. The network predicts the risk $h^j_\theta(\boldsymbol{X}_{(u_i,s_j)})$ for each product $s_j \in \mathcal{S}^{l+1}_{u_i}$ that was recommended in Stage I in Section III to user u_i. Thus, based on the assumption that each user has the same baseline Hazard function λ_0, it is possible to take the log of the Hazards ratio in order to compute the risk of each item s_j. Let us define the recommender function $rec^u_{i,j}(x)$ for user u_i as: $rec^u_{i,j} = log\left(\frac{\lambda_0 \cdot e^{h^i_\theta(\boldsymbol{X}_{(u,s_i)})}}{\lambda_0 \cdot e^{h^j_\theta(\boldsymbol{X}_{(u,s_j)})}}\right) = h^i_\theta(\boldsymbol{X}_{(u,s_j)}) - h^j_\theta(\boldsymbol{X}_{(u,s_j)})$.

The recommender function can be used to get the relationship between an item $s_i \in \mathcal{S}^{l+1}_{u_i}$ with another item $s_j \in \mathcal{S}^{l+1}_{u_i}$. When the function $rec^u_{i,j}$ is positive, the recommended item s_i leads to a higher risk of death than item s_j, so the item that should be recommended is s_j. Given the set $\mathcal{S}^{l+1}_u$, the recommender function $rec^u_{i,j}(x)$ for each $s_i, s_j \in \mathcal{S}^{l+1}_{u_i}$ is computed. Then, the recommended matrix $REC_u \in \mathcal{R}^{k\times k}$ is constructed as follows: $REC_u = \begin{bmatrix} rec^u_{1,1} & \dots & rec^u_{1,k} \\ \vdots & \ddots & \vdots \\ rec^u_{k,1} & \dots & rec^u_{k,k} \end{bmatrix}$.

The last matrix represents the relationship between the different items in the set $\mathcal{S}^{l+1}_u$. REC_u is an anti-symmetric matrix since it represents the difference of each risk $h^i_\theta({}_i\boldsymbol{X}_u)$ and $h^j_\theta({}_j\boldsymbol{X}_u)$. REC_u will be used to obtain the final recommendation s^*_u that has the lowest risk. For this aim, we search over all the elements in the matrix. For example, for each element $rec^u_{i,j}$ in the i-th row, if all of the values in that row are negative it means that the item s_i has the lowest risk. However, if the value $rec^u_{i,j}$ is positive, then the s_j gets a lower risk than s_i, meaning the search continues to the j-row and so on. To estimate s^*_u, we use the following equation: $s^*_u = arg\min_i\{\Lambda_{rec}\}$. First, we identify the i-th row that contains all the negative values. Then, the sum of each row is computed in $\lambda_{rec_i} = \sum_{j=1}^k rec^u_{i,j}$. Thus, the vector $\Lambda_{rec} = [\lambda_{rec_1}, \dots, \lambda_{rec_k}]^T$ is constructed. Finally, the arg min of Λ_{rec} is calculated and the result is s^*_u.

VI. Experimental Results

In this section we investigate the experimental results from our proposed stacked neural network. The dataset used belongs to a large Australian insurance company consisting of small-and-medium enterprises (SME) with transactions starting in 2007 and ending in 2019 that include 18 different items. The dataset had 19,174 customers and 173,127

Authorized licensed use limited to: Cambridge Institute of Technology - Bengaluru. Downloaded on March 12,2026 at 09:24:14 UTC from IEEE Xplore. Restrictions apply.

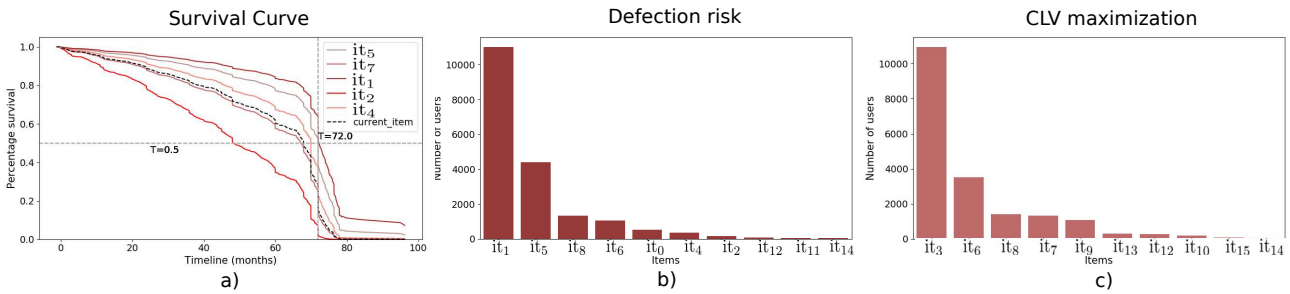

Fig. 2. a) User 06455's survival curves. The recommended product at the first stage is item it_5, the recommended product based on the defection risk is it_1 and the final recommended product that maximizes CLV is it_7. Top 10 items recommended based on b) defection risk and c) CLV maximization.

Stacked Neural Networks			Self-Attention Model				
User ID	MaxSurv	MaxCLV	top1	top2	top3	top4	top5
06455	it_1	it_7	it_5	it_7	it_1	it_2	it_4
11426	it_5	it_3	it_2	it_0	it_5	it_4	it_3
00002	it_1	it_6	it_5	it_4	it_6	it_0	it_1
06859	it_6	it_3	it_6	it_7	it_0	it_3	it_4
10969	it_4	it_3	it_0	it_7	it_6	it_4	it_3

TABLE I
PROPOSED STACKED NEURAL NETWORK VS. SELF-ATTENTION MODEL.

transactions. We used our stacked neural network model to obtain the item that maximize CLV. The top 5 recommended items, is obtained in stage I based on the self-attention recommender system. Then, Stage II computes the CLV using non-linear survival analysis and also obtains the recommended item that maximize CLV. The results are provided in Table VI. In the case of User 6455 the item that maximizes CLV the most is it_7, which was option three of the Self-Attention Model. Fig. 2 a) shows the survival function for user User 6455 for their current item and our 5 recommended items. Fig. 2 b) shows the top 10 recommended items based on defection risk. We can observe that item it_1 was the most recommended product. Fig. 2 c) describes the top 10 recommended products that maximize CLV in this case the it_3 was recommended the most.

VII. CONCLUSIONS

The system described was tested with data from a real insurance company. It was observed that the survival curves changed when varying products on the customers we simulated. The results show insurers can proactively focus their marketing efforts on more specific items that do increase customer lifetime value and divest on the ones whose cross-sell opportunities do not increase it. This technique manages to model the non-linear relationships of user's preferences through the neural network survival method, providing an edge when compared to other linear survival analysis methods in the literature. Another success was being able to factor in product features which most survival analysis methods do not consider in their time to event predictions.

ACKNOWLEDGEMENTS

The authors acknowledge financially support by Relativity6 Inc.

REFERENCES

[1] M. Lewis and N. Slack, *Operations management: critical perspectives on business and management*. Taylor & Francis US, 2003, vol. 1.

[2] B. P. Chamberlain, A. Cardoso, C. H. Liu, R. Pagliari, and M. P. Deisenroth, "Customer lifetime value prediction using embeddings," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2017, pp. 1753–1762.

[3] E. W. Ngai, L. Xiu, and D. C. Chau, "Application of data mining techniques in customer relationship management: A literature review and classification," *Expert systems with applications*, vol. 36, no. 2, pp. 2592–2602, 2009.

[4] J. L. Katzman, U. Shaham, A. Cloninger, J. Bates, T. Jiang, and Y. Kluger, "Deepsurv: personalized treatment recommender system using a cox proportional hazards deep neural network," *BMC medical research methodology*, vol. 18, no. 1, p. 24, 2018.

[5] P. S. Fader, B. G. Hardie, and K. L. Lee, ""counting your customers" the easy way: An alternative to the pareto/nbd model," *Marketing science*, vol. 24, no. 2, pp. 275–284, 2005.

[6] N. Glady, B. Baesens, and C. Croux, "A modified pareto/nbd approach for predicting customer lifetime value," *Expert Systems with Applications*, vol. 36, no. 2, pp. 2062–2071, 2009.

[7] X. Su and T. M. Khoshgoftaar, "A survey of collaborative filtering techniques," *Advances in artificial intelligence*, vol. 2009, 2009.

[8] M. J. Pazzani and D. Billsus, "Content-based recommendation systems," in *The adaptive web*. Springer, 2007, pp. 325–341.

[9] R. Burke, "Hybrid recommender systems: Survey and experiments," *User modeling and user-adapted interaction*, vol. 12, no. 4, pp. 331–370, 2002.

[10] Z. Lin, M. Feng, C. N. d. Santos, M. Yu, B. Xiang, B. Zhou, and Y. Bengio, "A structured self-attentive sentence embedding," *arXiv preprint arXiv:1703.03130*, 2017.

[11] Z. Yang, D. Yang, C. Dyer, X. He, A. Smola, and E. Hovy, "Hierarchical attention networks for document classification," in *Proceedings of the 2016 conference of the North American chapter of the association for computational linguistics: human language technologies*, 2016, pp. 1480–1489.

[12] C. Zhou, J. Bai, J. Song, X. Liu, Z. Zhao, X. Chen, and J. Gao, "Atrank: An attention-based user behavior modeling framework for recommendation," in *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.

[13] S. Zhang, Y. Tay, L. Yao, and A. Sun, "Dynamic intention-aware recommendation with self-attention," *arXiv preprint arXiv:1808.06414*, 2018.

[14] H. Zhang, I. Goodfellow, D. Metaxas, and A. Odena, "Self-attention generative adversarial networks," *arXiv preprint arXiv:1805.08318*, 2018.

[15] X. Chen, Y. Zhang, Q. Ai, H. Xu, J. Yan, and Z. Qin, "Personalized key frame recommendation," in *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2017, pp. 315–324.

[16] X. He, X. Du, X. Wang, F. Tian, J. Tang, and T.-S. Chua, "Outer product-based neural collaborative filtering," *arXiv preprint arXiv:1808.03912*, 2018.

[17] J. Ansell, T. Harrison, and T. Archibald, "Identifying cross-selling opportunities, using lifestyle segmentation and survival analysis," *Marketing Intelligence & Planning*, vol. 25, no. 4, pp. 394–410, 2007.

[18] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *2018 IEEE International Conference on Data Mining (ICDM)*. IEEE, 2018, pp. 197–206.

[19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *CoRR*, vol. abs/1706.03762, 2017. [Online]. Available: http://arxiv.org/abs/1706.03762

[20] S. Gupta and D. R. Lehmann, "Customers as assets," *Journal of interactive Marketing*, vol. 17, no. 1, pp. 9–24, 2003.

Authorized licensed use limited to: Cambridge Institute of Technology - Bengaluru. Downloaded on March 12,2026 at 09:24:14 UTC from IEEE Xplore. Restrictions apply.